Stock Market Prediction Using a Hybrid Approach With Boruta and Liquid Neural Network

G. Joshita Reddy, Rahul Kumar Sahani, and S. P. Raja

***Abstract*—Globally, one of the most fundamental applications of time-series data is stock market forecasting, where each and every second is crucial and analysis is unpredictable, posing a significant challenge towards predicting its trend. Various research is underway for the analysis of the stock market using various time series algorithms such as long-short-term memory (LSTM), recurrent neural network (RNN). Here, in this article, we have proposed a novel approach of multiple layers including liquid time constant, the linear first order dynamics with nonlinear interlinked gates which improves performance of time series; after which we included liquid neural network and forecasted the results. Then, we enhanced the algorithm using Boruta feature selection, where we have selected only the required columns and ranked them as per requirement. Furthermore, for more accurate results, we have selected three different sets of historical data for five large cap sectors (Alphabet, Apple, Blackrock, JP Morgan, and IBM) after which we observed the R-square values along with all the performance evaluation and error evaluation results for all the individual dataset in two phases, i.e., before and after using the Boruta feature selection method for 5, 10, and 15 years data, we observed the best results for LSTM (0.9443) and RNN (0.9752). Liquid neural networks, being the most efficient and accurate, gave the best R-square value of 0.9835.**

***Index Terms*—Boruta algorithm, liquid neural networks (LNNs), liquid time constant (LTC), stock market prediction.**

I. Introduction

MODERN stocks have actually been around since 1602 when they were first invented in the country of Amsterdam, Netherlands where people began trading for what is now known as the Dutch East India Company. Originally, trading was restricted to this single organization, with the first derivative transactions taking place in 1607 followed by profit sharing through dividends some years later. The stock market is a place where shares are transformed, sold and transferred. It provides an important avenue for large sectors to gain capital from public (shareholders). After issuance, a huge amount of monetary capital is injected into this financial market, making it the formation of commercial asset structure by stimulating investors' interest in investing. This has made great contributions to product economic growth.

The circulation of stocks allows capital to be distributed, which in turn promotes growth in investments. Political atmosphere, finance and monetary situation, market environment, technological progressions and shareholder state-of-mind are among the reasons that trigger variations of stock indices in multiple directions. The consequence, the prices of stock keep on changing, thus provisioning a reason for speculative activities and again enhancing the level of uncertainty pertaining to these indices. The volatility is not only a financial risk for shareholders, it also may act as a constraint on the economic development of corporations and nations. Therefore, an accurate analysis and predication of stock indices could play a focal role in the decision making process of stakeholders as well help them to keep up with regards to their economic insatiability.

Once you have to analyze the financial markets, such as collecting, arranging, and lining up relevant data from them will enable you both to comprehend where an index is going next at any specific time. Therefore, it will keep shareholders' losses at a minimum and also supports in earning profits. As a result, the methodological analysis and forecast of the stock market will provide shareholders with every writer ted profit, because indices are inherently unpredictable, forecasting them continues to be a difficult problem in financial time series analysis. The significance of financial economics study is often emphasized in research publications, given the uncertainty that shareholders seeking to maximize profits face.

Time series analysis has historically relied on established techniques for predicting indices, which emphasizes the importance of ongoing research in this field. Predicting future price fluctuations on the stock market requires analyzing historical data. Even while they have their uses, traditional approaches frequently fail to capture the nonstationary and nonlinear characteristics of financial markets. Because of their ability to handle information dynamically and adaptively, liquid neural networks (LNNs) present a possible substitute.

Liquid neural networks (LNNs) are inspired by the adaptability of biological neural circuits. They are a variant of recurrent neural networks, which can dynamically adjust to new inputs. This makes them suitable for forecasting trends in stock market,

Received 16 September 2024; revised 7 January 2025; accepted 5 February 2025. Date of publication 20 February 2025; date of current version 2 October 2025. *(Corresponding author: S. P. Raja.)*

The authors are with the Department of Computer Science, Vellore Institute of Technology, Vellore 632014, India (e-mail: joshita9reddy@gmail.com; kilopsahani2@gmail.com; avemariaraja@gmail.com).

Digital Object Identifier 10.1109/TCSS.2025.3540035

which are known to be inherently volatile and governed by a multitude of factors. A LNN is modeled as an ensemble of nonlinear units (a set $N \in 1, 2, \ldots, n$) interconnected with $N \times n$ weighted connections Y_{ij}.

The major contributions of this article are outlined as follows.

1) Boruta algorithm, a resilient feature selection method that works on 'all-relevant' principle, is utilized to determine the most suitable features in the stock market datasets for training.
2) Liquid time constant (LTC) identifies the rate at which information dissipates over time within the liquid layer in LNN and thereby improves the stock market predictions.
3) Revolutionary type of neural networks, LNN, are used to analyze both short-term trends and long-term dependencies in the stock markets, offering unparalleled flexibility and efficiency in temporal data processing.
4) Extensive results validated on the latest stock data to show the superior performance of the Boruta algorithm with LNN compared to other algorithms.

This article is structured as follows: Section II surveys the existing state of literature in the realm of stock market prediction; Section III expounds our proposed methodology along with other state-of-art methods we have compared our model with; Section IV describes the experimental results and discusses the insights obtained from them; and Section V concludes the article while highlighting prospective future works.

II. Related Works

P. Gajjar et al. [4] endeavored to utilize LTC networks in stock market trading, demonstrating their rapid ability to adapt to fluctuating market conditions. K. Uhle [3] explores the resilience of neural networks in predicting stock liquidity in current stock markets. X. Pang et al. [9] propose an innovative approach to predicting stock with neural networks, where they introduced "stock-vector," a high-dimensional stock market data. M. Kumbure et al. [18] inspects the working of various machine learning algorithms for stock market forecasting. By exploiting a range of machine learning techniques, including regression, classification, and ensemble methods, they identify effective models for predicting stock market fluctuations.

J. Li et al. [15] showcases the application of the renowned feature selection method, Boruta algorithm, along with adaptive denoising for stock price prediction. Boruta algorithm is utilized to identify relevant features from a large set of potential predictors, helping in the building of more accurate forecasting models. M. Sethi et al. [23] employed this feature selection algorithm to analyze Twitter sentiment and predict stock prices. This study showcased the efficiency of Boruta in filtering relevant features for stock market prediction.

J. Ohana et al. [5] utilized explainable AI in order to emphasize on making AI models more transparent and interpretable, thereby, grasping the decision-making processes in stock market analysis. T. Celik et al. [6] also proposed a comprehensive framework for assessing interpretability in machine learning models. Y. Fukuchi et al. [7] introduced a unified approach to interpret model predictions, boosting the explainability of AI in finance. Freeboroug et al. [8] debates on the obstacles and misunderstanding in model interpretability, providing further perception on the application of XAI in financial models.

J. Chen et al. [10] elaborates on model interpretation, taking an information-theoretic approach applicable to financial models. Y. Li et al. [11] couples machine learning algorithms with SHAP values in order to provide interpretable stock market predictions. A. Agarwal et al. [12] illustrates the use of LIME and SHAP to elucidate financial foretelling. S. Gite et al. [13] presents hybrid model combining neural networks and XAI, and achieved high accuracy and interpretability in stock prediction.

Long short-term memory (LSTM) networks and recurrent neural networks (RNN) are renowned for their capability to capture time-related dependencies. Ramin et al. [14] introduced a new class for time-continuous RNN models where they constructed a network of first order dynamic system modulated through nonlinear interlinked gates which exhibited stable and bounded behaviors for time series prediction tasks. R. Chiong et al. [1] proposed a novel approach using ensemble-RNN coupling sentiment evaluation with Sliding-Window technique which outperform traditional RNNs and LSTMs in encapsulating time-series based dependencies and complex trends in stock data.

Khatri et al. [46] also used RNN architecture to examine Twitter data and predict stock prices. The high potential of RNN in capturing compounded trends in stock market data was highlighted in the results. W. Chen et al. [24] proposed RNN-boost to analyze Twitter data related to stocks. The study also highlighted the success of RNN in modeling the long-term time dependencies in stock market data. S. Wu et al. [26] also proposed sentiment analysis using RNN to foretell stock market trends. The study showed the high likelihood of using RNN in modeling sentiment-based patterns. Y. Liu et al. [27] leveraged RNNs to predict stock prices based on financial news and microblog data. These studies feature the effectiveness of such algorithms in providing more precise prediction.

K. Pawar et al. [16] studies LSTM for predicting stock returns. LSTM networks are also a category of RNN, that can capture time-based dependencies in continuous data. This makes them suitable for stock-based time series prediction. D. Nelson et al. [17] also scrutinizes the application of LSTM networks for temporal prediction tasks. Long-term dependencies in uninterrupted data, can be learnt LSTM networks, which is necessary for modeling the complex patterns present in financial time series.

H. Widiputra et al. [19] suggests a unique strategy for stock forecasting by using a fusion model merging LSTM and convolutional neural networks (CNNs). By combining various representations of the same data, the approach aims to model diverse features and boost predictive performance. F. Moodi et al. [28] also utilized CNN-LSTM to foretell the trend of stock market. X. Zhang et al. [21] implemented an attention based LSTM for analysis of stock news and predict stock prices. This study demonstrated the virtue of LSTM in uncovering long-term dependencies in stock data. B. Pramod et al. [25] implemented

TABLE I
DATASET DESCRIPTION

Company	Years	Start Date	End Date	No. of Samples
Alphabet (GOOG), Apple (AAPL),	5	24-05-2019	24-05-2024	1260
BlackRock (BLK), JPMorgan (JPM),	10	24-05-2014	24-05-2024	2518
IBM (IBM)	15	24-05-2009	24-05-2024	3777

LSTM to analyze textual data for predicting stock prices. The results indicated that LSTM was effective in capturing intricate patterns in stock market data.

Hybrid models incorporating various ML techniques are known to be quite promising in enhancing the model performance. Zhong and Enke [34] used hybrid machine learning algorithms to forecast daily stock market return directions, leveraging their strengths and diminishing individual weaknesses by combining different methods. A combination of LSTM and RNN was wielded by Hansun et al. [20] in order to predict stock prices based on financial news and technical features. The results convey noteworthy improvements in prediction compared to conventional methods.

Kumar et al. [37] implemented metaheuristic optimization on LSTM-RNN networks for boosted intra-day stock market forecasting, demonstrating the benefits of optimization in fine-tuning of the model parameters. The utilization of real-time data has been noted to be necessary for improving the reliability of stock market predictions. Md et al. [30] proposed a multilayered sequential LSTM model for stock price forecasting, mentioning its superiority in handling time-related data, and discovering complex trends [30]. Similarly, Yu et al. [33] and Gao et al. [41] implemented convolutional recurrent neural network (CRNN) with LSTM architectures for trend prediction, illustrating remarkable developments in the field.

S. Li et al. [2] apply GARCH models to capture stock volatility, achieving high performance compared to other traditional approaches. Mahajan et al. [43] modeled and forecasted the volatile nature of the NIFTY-50 index with the help of RNN and GARCH models, thereby, explaining the significance of volatility factor in commercial decision-making. J. Nielsen et al. [22] used deep log-periodic power law singularity strategy to foretell stock prices based on financial indicators. The results showed that it outperformed traditional methods in terms of various metrics. Rundo et al. [40] inspected the applications in machine learning of quantitative finance, covering a broad range of methods and their economic applications.

Some of the studies focus on certain markets or indices. To predict daily prices of stock in the Sri Lankan financial market, Samarawickrama et al. [35] used RNN, hence, displaying the flexibility of ML techniques across different environments. Novel approaches continue to arise in this dynamic field. A combined feature-reduction technique employing recursive feature elimination and variational auto-encoders was presented by Gunduz [42] aiding in boosting effectiveness of the model. Sheth and Shah [31] deliberated over the top practices and ways to predict stock markets with the help of ML, focusing on their continuous advancement and improvement.

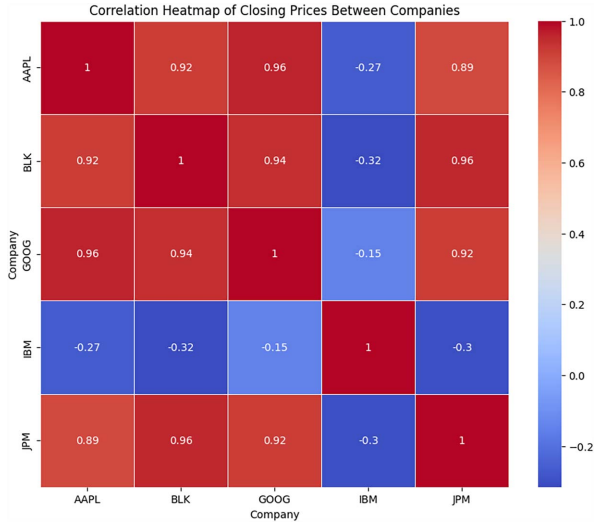

Fig. 1. Correlation heatmap of closing prices between companies.

Strader et al. [32] discussed various stock market prediction studies based on machine learning, highlighting the pros and cons of this field. Albahli et al. [38] built a machine learning method which leverages real-time Twitter data to predict stock market trends, illustrating the necessity of real-time sentiment data on stock market forecasts. Khalid et al. [29] explored sentiment classification based on unstructured reviews with the help of an ensemble classifier approach called GBSVM. The study demonstrated the effectiveness of aggregating multiple classifiers to heighten sentiment analysis efficiency. Comprehensive reviews and comparative studies offer cardinal intuition into the efficacious of various strategies in stock market prediction.

III. METHODOLOGY

A. Dataset Description

The stock market data of five companies viz. Alphabet, Apple, BlackRock, JPMorgan, and IBM were extracted from Yahoo Finance [47]. These data are collected from the past 5, 10, and 15 years as shown in Table I. The data contains features describing the stock market, namely, Open, Close, High, Low, Adjusted Close, Day, Date, and Year.

The stock market data are analyzed and visualized for trends, distributions, and relationships in stock prices to uncover patterns. Fig. 1 visualizes the correlation matrix of closing prices between companies. This matrix depicts the degree and direction of relationships between companies' stock prices. For example, strong positive correlations are observed between technology firms Apple, Google, and BlackRock, while IBM

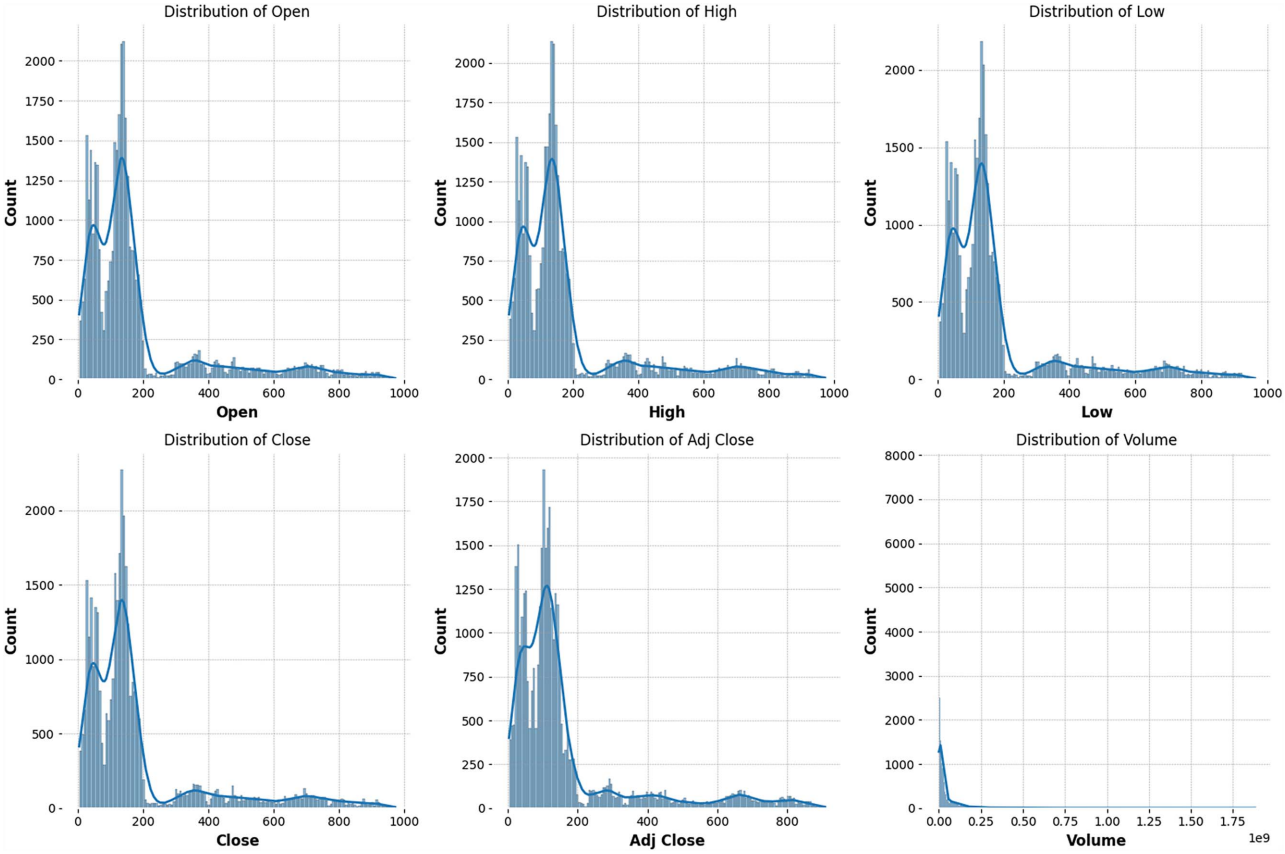

Fig. 2. Distribution of stock market features across companies.

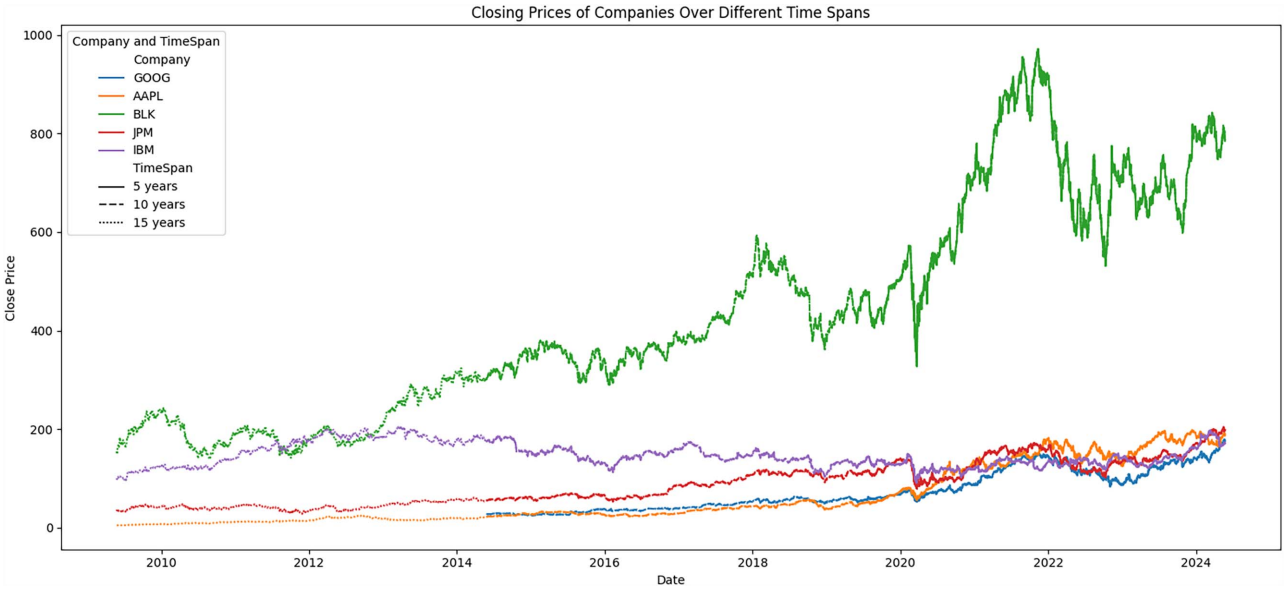

Fig. 3. Closing prices of companies over different time spans.

exhibits weaker correlations. Fig. 2 shows histograms representing the distribution of numerical features in the stock market data, highlighting varying ranges and frequencies across different companies and time spans.

Fig. 3 highlights the trend in closing prices over 5, 10, and 15 years for multiple companies helping in visualizing temporal patterns and comparing performance across companies. The trends in closing prices over different time spans can reveal

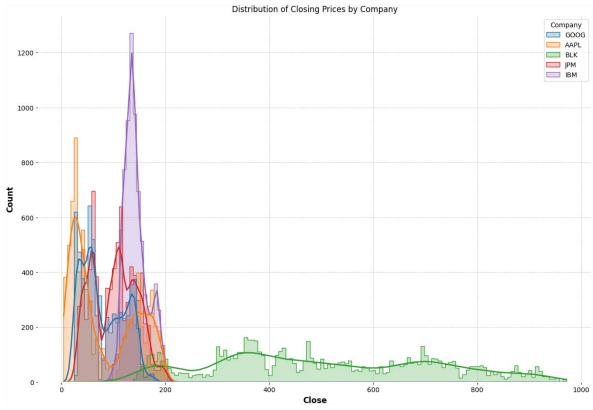

Fig. 4. Distribution of closing prices by company.

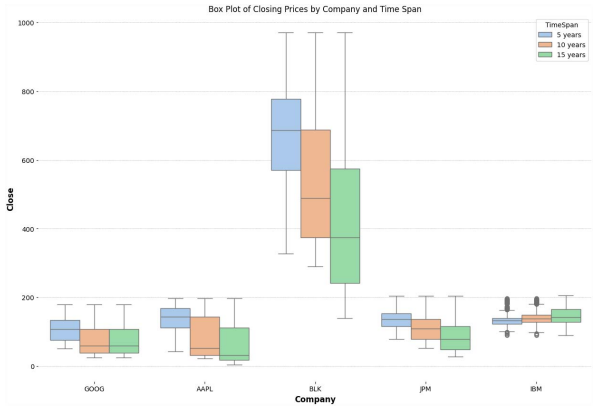

Fig. 5. Box plot of closing prices by cand time span.

how companies have performed in the long term. In Fig. 4, the frequency distribution of closing prices for each company is depicted, giving insights about central tendency and variability of prices within each company's stock data. The spread of the distribution reflects the variability in stock prices. A wide distribution such as BlackRock's indicates high price variability, while a narrow distribution suggests more stable prices. This can help investors understand the risk associated with each stock.

The box plot in Fig. 5 visualizes the spread of closing prices for each company across various time spans, highlighting potential differences in price distributions. The box plot displays the median, quartiles, and possible outliers for the closing prices of each company. The length of the box represents the inter quartile range (IQR), which indicates the spread of the middle 50% of the data.

B. System Architecture

Stock market is a highly volatile field in terms of the dynamic nature of the stock prices, and hence, it is extremely necessary to make predictions with high accuracy and efficiency in order to avoid monetary loss. Our work proposes a hybrid novel approach that couples the resilient feature selection method known as the Boruta algorithm with LTC and liquid neural to capture the short-term and long-term dynamics of stock market data with high efficiency and accuracy. After stock market data collection, we bring the training data to uniform range with help of min–max normalization which is defined by the equation

$$s' = \frac{s - s_{\min}}{s_{\max} - s_{\min}}. \tag{1}$$

Here, s is the original value, s' is the normalized value, $s_{\min}$ is the smallest value in the dataset, and $s_{\max}$ is the largest value in the dataset. Afterwards, the Boruta algorithm is applied to select the features which are most relevant while making the prediction. Then, the liquid time constant is used to model the dynamics of neurons with the aid of differential equations. The selected parameters are used to train the LNN and the results are obtained are verified with the help of testing data as depicted in Fig. 6.

C. Boruta Algorithm

Feature selection is an essential process of developing machine learning architectures, especially for datasets of higher dimensions such as the ones used for stock market prediction. Unlike traditional methods focusing on finding a small subset of features, Boruta is an all-relevant feature selection algorithm which aims to discover every feature which contribute towards prediction ability of a machine learning model. This meticulous method guarantees that no helpful feature is missed.

Boruta algorithm utilizes a random forest classifier, a distinguished and resilient technique capable of handling huge data. The algorithm evaluates the importance of each original feature with respect to a shadow feature, that is to say its randomly shuffled copy. This contrast helps in dictating each feature's importance. The initial procedure consists of creating shadow features via randomly permuting the values of the original features. This effectively ruins any relationship they might have with the target variable, the Algorithm 1 shown the working of the model.

Using a dataset consisting of both the original and shadow features, a random forest classifier is trained. Then, we measure the importance of each feature with the help of the mean decrease in impurity (MDI), also known as the Gini Importance. Mathematically, Boruta feature selection involves calculating the importance score of a feature using the MDI. The impurity decrease $\Delta I(X_j, m)$ at node m in a decision tree is given by

$$\Delta I(Z_k, m) = I(m) - \left(\frac{N_{\text{left}}}{N} I(m_{\text{left}}) + \frac{N_{\text{right}}}{N} I(m_{\text{right}}) \right) \tag{2}$$

where $I(m)$ denotes the impurity at node m, N is the number of samples at node m, N_{left}, and $I(m_{\text{left}})$ represents the number of samples and impurities in the left child node after the split, and N_{right} and $I(m_{\text{right}})$ represents the number of samples and impurities in the right child node, respectively. The total importance score for feature Z_j is then

$$\text{Importance}(Z_j) = \frac{1}{T} \sum_{t=1}^{T} \sum_{m \in \mathcal{N}_t} \Delta I(Z_j, m). \tag{3}$$

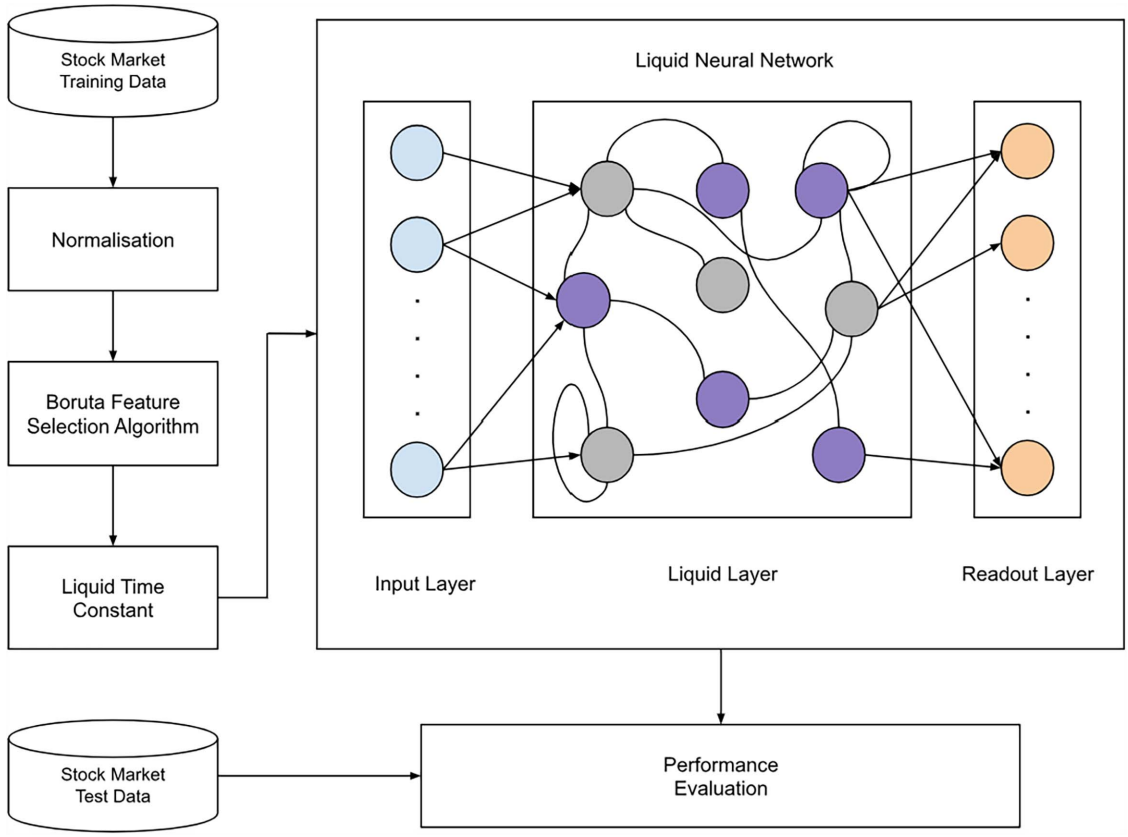

Fig. 6. System architecture.

Algorithm 1: Boruta Algorithm.

Input: Stock data D with Parameters P and target value C
Output: Set of included Parameters P_{include}
Initialize $P_{\text{Include}} \leftarrow \emptyset$
repeat
  Create shadow features S_P by shuffling the original Parameters P
  Training random forest classifier on the Stock data begin with S_F
  Calculate valuable scores for P and S_P
  for each parameter p in P **do**
    if valuable(p) $>$ max(valuable(S_P)) **then**
      Add p to P_{include}
    end if
  end for
  Remove Parameters frequently less important than shadow features
until convergence
return F_{include}

Here T denotes the number of trees, $\mathcal{N}_t$ is the collection of nodes in tree t, and $\Delta I(Z_j, m)$ represents the decrease in impurity resulting from splitting on feature Z_j at node m. Later on, we compare the importance of each original feature with the highest importance score achieved among the shadow features. Then, the features are categorized as follows.

1) Confirmed Important: If the importance of the original feature is significantly higher as compared to the highest importance amongst the shadow features.
2) Rejected: If the importance is significantly lower as compared to the highest importance amongst the shadow features.
3) Tentative: If there exists no difference between the maximum importance amongst the shadow features.

This iterative procedure stops under two conditions: 1) every feature are either confirmed or rejected and 2) predefined number of iterations is reached. This thorough approach ensures the selection of features that are actually contribute towards the prediction task. This boosts the robustness and performance of any model, especially in the context of stock market prediction where data are high-dimensional and relationships are complex and non-linear.

D. LTC

The LTC is a significant parameter to manage the temporal dynamics of neural networks which are developed to handle time-series data, such as stock market prediction models. This parameter determines the rate of information dissipation over time within the liquid layer or dynamic reservoir layer in liquid state machines (LSMs) and LNNs.

Algorithm 2: Liquid Time Constant.

Input: Time-series stock data X, initial parameters W, time constant τ
Output: Forecasted stock prices Y
Set initial liquid state h_0
for each time stamp t **do**
  Calculate state change: $\delta h_t \leftarrow \frac{1}{\tau}\big(-h_{t-1} + f(W, h_{t-1}, x_t)\big)$
  Update liquid state: $h_t \leftarrow h_{t-1} + \delta h_t$
  Determine output: $y_t \leftarrow g(h_t)$
end for
return Y

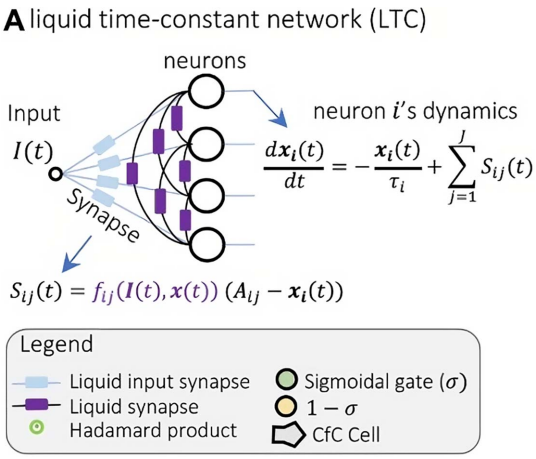

Fig. 7. LTC network. (M Chahine et al. [45]).

A LSM or LNN consists of a reservoir of neurons with recurrent connections, which process time-fluctuating inputs. The state $x(t)$ of the liquid layer evolves as per the following differential equation

$$\tau \frac{ds(t)}{dt} = -s(t) + F(s(t), u(t)). \tag{4}$$

Here, τ is the time constant, $s(t)$ represents the network's state at timestamp t, F symbolizes the nonlinear function depicting the system's dynamics, and $u(t)$ is the external input. The time constant τ determines how rapidly the network state reacts to variations in the input. Algorithm 2 shows how the liquid state changes and the updated. A smaller value of τ results in faster adaptation, which is suitable for capturing rapid fluctuations in the input data. Conversely, a larger value of τ provides more stability and smooths out short-term variations.

The LTC is shown in Fig. 7. The behavior of an individual neuron i is described, where $\mathbf{x}i(t)$ denotes the neuron's state at time t and τ signifies neuronal time constant. Synaptic connections are characterized by a nonlinear-function, with $f(.)$ representing a sigmoidal nonlinearity. A_{ij} stands for the reversal-potential parameter for every synapse from neuron j to neuron i. It is quite crucial in predicting the state of stock market, where trends in financial data can change notably over different time scales. The predictive ability of the model is enhanced due to its potential to adjust the time constant allowing the model to uncover both short-term volatility and long-term patterns.

E. LNN

Building on the ideas already present in the LSM, a LNN is a network that serially processes and forecasts time-varying input combining a dynamic reservoir which is a liquid layer with an readout layer. A LNN architecture consists of three main parts: the input layer, the liquid (reservoir) layer, and the readout layer. The input layer accepts the external input signals ($u(t)$), such as historical stock prices, trading volumes, and other financial indicators for stock market prediction. This is then set as input for the layer of liquid which is just a bunch of interconnected neurons with recurrent connections. The liquid layer is defined by its state as

$$\tau \frac{dx(t)}{dt} = -x(t) + W_{\text{in}} u(t) + W_{\text{rec}} \sigma(x(t)). \tag{5}$$

In this equation, W_{in} is the input weight matrix, W_{rec} is the recurrent weight matrix, and $\sigma(\cdot)$ represents a nonlinear activation function such as sigmoid or tanh. The state $x(t)$ encapsulates the dynamic response of the liquid layer to the input signals and its own past states. This differential equation can be discretized for numerical simulation using the Euler method. For a small time step Δt, the discrete-time update rule is:

$$x(t + \Delta t) = x(t) + \frac{\Delta t}{\tau}\left(-x(t) + W_{\text{in}} u(t) + W_{\text{rec}} \sigma(x(t))\right). \tag{6}$$

The readout layer maps the dynamic states of the liquid layer to the desired output, computed as

$$y(t) = W_{\text{out}} \sigma(x(t)) \tag{7}$$

where W_{out} is the readout weight matrix.

More generally, the parameters (W_in}), (W_rec}) and (W_out}) are learned from data. During training, the model learns the parameters such that the discrepancy between forecast and true values (as captured by the loss function) is minimized. The readout layer is normally trained in a supervised manner to minimize a loss function, e.g., mean squared error similarity (MSE) for regression tasks or cross-entropy loss for category tasks.

The LNN architecture is quite well-matched for time series prediction. The liquid layer's dynamic states capture intricate temporal dependencies in the input data which are essential for accurate stock market predictions. Furthermore, the separation of the liquid layer and the readout layer simplifies the training process. The liquid layer acts as a dynamic feature extractor, while the readout layer, being linear, can be trained efficiently using standard optimization algorithms, we have presented the algorithm for liquid neural network in two phases: Algorithm 3 shows the basic working and Algorithm 4 shows the detailed working of LNN.

For instance, in predicting the future price of a stock, the input layer receives a time series of historical prices and trading volumes. The liquid layer processes these inputs, capturing the hidden temporal patterns. The readout layer then uses these

Algorithm 3: Liquid Neural Network for Stock Market Forecasting.

Input: Time-series stock data X, starting weights W, and hyperparameters α, γ
Output: Predicted stock prices P
Initializing liquid state $h_0 \leftarrow 0$
Initializing synaptic weights W_h, W_x, W_y
for each training epoch **do**
  for each time step t **do**
    Input Integration:
    $u_t \leftarrow W_x \cdot x_t + W_h \cdot h_{t-1}$
    Updating States:
    $h_t \leftarrow (1-\alpha) \cdot h_{t-1} + \alpha \cdot \tanh(u_t)$
    Calculation of output:
    $y_t \leftarrow W_y \cdot h_t$
    Calculation of errors:
    $e_t \leftarrow y_t - y_t^{\text{true}}$ {True value of the stock price at specific time t}
    Updating the weights: {Using gradient descent}
    $W_y \leftarrow W_y - \gamma \cdot \frac{\partial e_t}{\partial W_y}$
    $W_h \leftarrow W_h - \gamma \cdot \frac{\partial e_t}{\partial W_h}$
    $W_x \leftarrow W_x - \gamma \cdot \frac{\partial e_t}{\partial W_x}$
  end for
  End of Epoch: Adjust the learning rate wherever necessary
end for
return P

Algorithm 4: State and Weight Transition Mechanisms.

Integrating the input:
$u_t \leftarrow W_x \cdot x_t + W_h \cdot h_{t-1}$
u_t is the aggregate input from the current input x_t and the previous state h_{t-1}.
Updating the states:
$h_t \leftarrow (1-\alpha) \cdot h_{t-1} + \alpha \cdot \tanh(u_t)$
The new state h_t is a aggregation of the previous state and the non-linear transformation of the insert integration u_t, modulated by a mixing factor α.
Calculating the output:
$y_t \leftarrow W_y \cdot h_t$
The output y_t is computed by a linear transformation of the current state h_t.
Calculating the error:
$e_t \leftarrow y_t - y_t^{\text{true}}$
The error e_t is the distinction between the predicted output y_t and the true stock price y_t^{true}.
Updating the weights:
$W_y \leftarrow W_y - \gamma \cdot \frac{\partial e_t}{\partial W_y}$
$W_h \leftarrow W_h - \gamma \cdot \frac{\partial e_t}{\partial W_h}$
$W_x \leftarrow W_x - \gamma \cdot \frac{\partial e_t}{\partial W_x}$
The weights W_y, W_h, and W_x are updated using gradient descent with learning rate γ.

patterns to predict the future price, making it a potent tool for making informed investment decisions.

F. RNNs

RNNs are neural networks that are proficient in dealing with sequential data. Unlike feed-forward neural networks, RNNs have connections that develop directed cycles, which allows them to maintain a hidden state to capture details about past elements in the sequence. This makes them especially useful for tasks like time-series prediction, Natural language processing, and speech-recognition, where the input has a temporal or sequential form.

RNN processes a sequence of inputs $\{x_t\}$ and maintains hidden state s_t which is revised at each time stamp. This hidden state stores the details regarding the past elements in the sequence. The update rule for the hidden state s_t, and the output y_t at time stamp t is described as follows:

$$s_t = \sigma(U_h.h_{t-1} + W_h.x_t + b_h) \tag{8}$$

$$y_t = \phi(W_y h_t + b_y). \tag{9}$$

Here, hidden state at time stamp t is given by s_t, x_t, and y_t represents input and output at time stamp t. W_h, U_h, and W_y are weight matrices The bias vectors are given b_h and b_y in the equations and σ represents activation function (e.g., $\tanh$ or ReLU). ϕ is the output activation function

The hidden-state s_t at each time stamp t depends on the input x_t at that time stamp and the hidden-state s_{t-1} from the previous time stamp. This recursive relation lets the network to store the information from previous values, effectively giving it memory. In Algorithm 5, at each time stamp t, the hidden-state s_t is updated using the present input x_t, and the past hidden-state s_{t-1}, and y_t output is computed.

Algorithm 5: Recurrent Neural Network (RNN).

Input: Time-series data X, starting weights W
Output: Predicted stock prices Y
Initializing hidden state s_0
for every time stamp t **do**
  Update hidden state: $s_t \leftarrow \tanh(W_h \cdot s_{t-1} + W_x \cdot x_t)$
  Compute output: $y_t \leftarrow W_y \cdot s_t$
end for
return Y

G. LSTM

LSTM networks, are a sort of architecture that extends the RNN. It is more capable of unearthing the long-term dependencies and can even tackle the vanishing gradient problem. Unlike regular RNNs, which have problem to find out anything longer than concerning 10 to 20 consecutive time actions, LSTMs don't deal with this predicament. LSTMs are able to learn to recognize patterns in sequences of events lasting across hundreds, or even thousands, of time steps. These cells have gates: 1) input gate which gate helps us decide what and when to store new information; 2) forget gates gate helps decide when to forget; and 3) output gates gate regulates what the read out

Algorithm 6: Long Short-Term Memory (LSTM).

Input: Time-series data X, starting weights W
Output: Predicted stock prices Y
insert hidden state s_0 and cell state c_0
for every time stamp t **do**
  ForgetGate: $f_t \leftarrow \sigma(W_f \cdot [s_{t-1}, x_t] + b_f)$
  InputGate: $i_t \leftarrow \sigma(W_i \cdot [s_{t-1}, x_t] + b_i)$
  CandidateCellState: $\tilde{c_t} \leftarrow \tanh(W_c \cdot [s_{t-1}, x_t] + b_c)$
  refurbish CellState: $c_t \leftarrow f_t \cdot c_{t-1} + i_t \cdot \tilde{c_t}$
  OutputGate: $o_t \leftarrow \sigma(W_o \cdot [s_{t-1}, x_t] + b_o)$
  refurbish HiddenState: $s_t \leftarrow o_t \cdot \tanh(c_t)$
  Calculate Output: $y_t \leftarrow W_y \cdot s_t$
end for
return Y

is going to be. They present an intricate architecture with gates that regulate the flow of information.

LSTMs plays a major role in the domain of stock market forecasting, The network takes the sequential data one step at a time, updating its internal state after each timestamp, and then uses that updated state to make another prediction. The forget gate f_t tells which information from the previous cell state c_{t-1} should be eliminated. The input gate i_t dictates which new information from the candidate cell state $\tilde{c_t}$ should be added to the cell state. The cell state c_t is later updated with this new information. The output gate o_t determines which information from the cell state c_t should be output as the hidden state h_t as shown in Algorithm 6. Thus, LSTM networks work well in learning the complex and nonlinear relationships in financial time series data, making a very powerful tool for forecasting financial stock market movements.

Here is a concise mathematical model for LSTM

$$
\begin{aligned}
f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\
i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\
o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\
\tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\
C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\
h_t &= o_t \odot \tanh(C_t)
\end{aligned}
$$

where f_t is the forget gate, i_t is the input gate, o_t is the output gate, $\tilde{C}_t$ is the candidate cell state, C_t is the cell state, h_t is the hidden state, σ is the sigmoid activation function, $\tanh$ is the hyperbolic tangent activation function, and $\odot$ is element-wise multiplication.

The main elements include gates that regulate the information flow, the cell state which preserves memory, and the hidden state responsible for generating the output. These components are computed and revised over time according to specific equations.

H. Application to Stock Market prediction

Integrating Boruta feature selection with LNNs has high potential for predicting stock market trends. A stock market is distinguished by its highly volatile nature, nonlinear trends, and complex inter-dependencies between different financial features are characteristics of the stock market. Traditional models often fail to account for these complexities, and hence give suboptimal predictions. The combination of Boruta and LNNs offers a thorough framework to overcome these problems.

Boruta algorithm makes sure the model utilizes all pertinent information filtered out from the past prices, trade volumes, and other economic factors. It filters out the features that play a distinguishing role in prediction. By eliminating noisy and irrelevant features by meticulously testing against their randomly permuted shadow counterparts, Boruta feature selection boosts the robustness of the model. LNNs with their dynamic reservoirs, are experts at capturing temporal dependencies in stock market data. Governed by both external inputs and its recurrent dynamics, the liquid layer's state evolves over time, thereby, allowing it to model short-term and long-term trends effectively. This temporal modeling capability is quite important for forecasting future stock prices.

Let's consider a scenario where we need to foretell the upcoming price of a stock. The input layer of the LNN receives time-series historical price data and trading volumes. These inputs are then sent and processed by the liquid layer, thereby, capturing the underlying temporal trends. The readout layer then maps these trends to a predicted future price. By continuously updating the state of the liquid layer as new data arrives, the LNNs can provide real-time predictions, making it a highly valuable tool for traders and investors.

Boruta algorithm ensures that all relevant information is sustained while rest are discarded, while the LNNs leverages the dynamic properties of its liquid layer to model complex temporal dependencies. Hence, our approach enhances the performance of the model, providing accurate and timely predictions that can inform investment decisions as well as mitigate financial risks. The mathematical exactness and versatility of these methods make them well-matched for the challenging task of predicting stock market behavior as well as paving a way for more innovative financial forecasting models. A solid and scalable implementation pipeline is required to integrate proposed stock market prediction model with Boruta and LNNs in the real world systems. It all starts with a solid data collection infrastructure—interfacing with APIs like Bloomberg or Yahoo Finance to pull real time and historical stock prices, trading volumes, and macroeconomic indicators. For example, data preprocessing should include basic tasks like dealing with missing values, eliminating outliers, and normalizing features, which make input clean and consistent. Using containerization techniques like Docker to provide scalability and dependability, the model can be implemented on cloud platforms like AWS or Azure for real-time predictions. Low-latency forecasts made possible by a REST API can give traders and analysts useful information. While fallback measures, such as switching back to simpler models, help control risks during unpredictable market situations, performance monitoring systems should track important parameters like prediction accuracy and response time. A user-friendly interface, like a web dashboard, can show trend visualizations, feature relevance, and predictions, enabling end

users to access the system. To maintain confidence and openness, security measures—such as data encryption and adherence to financial regulations—are essential. These components can be used to successfully apply the model and provide real-time, accurate, and actionable stock market data. Designed for that, Boruta can be implemented to dynamically choose out features which are more relevant and remove noise and redundant variables. This effectively increases the efficiency and accuracy of the LNN. The LNN learns on this refined dataset to model nonlinear relationships and familiarize itself with the constantly changing trends of the stock market. Re-training both Boruta and the LNN regularly keeps the model tuned to evolving market conditions. The fitness and advantages of Boruta for stock market prediction should be clarified. The Boruta algorithm is well-suited for stock market prediction due to its ability to manage complex, high-dimensional datasets often associated with financial markets. These datasets typically include various features such as historical stock prices, economic indicators, sentiment data, and market indices. Boruta effectively identifies critical predictors from these datasets while filtering out irrelevant ones, ensuring that all significant features are retained. Its robustness to noise, a common challenge in financial data, is another strength, as the algorithm uses shadow features (randomized versions of actual features) to reliably evaluate their importance. Additionally, Boruta's flexibility allows it to integrate seamlessly with various models, including machine learning frameworks like random forests, gradient boosting, and deep learning algorithms. The advantages of Boruta are numerous and valuable in stock market forecasting. It ensures the inclusion of all relevant features, even those with secondary importance, thereby improving the accuracy and reliability of predictive models. By removing unnecessary variables, it minimizes the risk of overfitting, which is a frequent problem in financial modeling due to the complexity and noise in the data. The algorithm's categorization of features into relevant, irrelevant, and tentative groups makes its output highly interpretable, simplifying feature selection processes. Moreover, its automation and scalability enable it to handle large datasets efficiently with minimal human effort, making it an excellent tool for real-world applications where data size and complexity are significant challenges.

IV. Results and Discussions

A. Evaluation Metrics

The following metrics collectively provide a comprehensive overview of model performance, highlighting different aspects such as accuracy, bias, and variance. This helps in ensuring the model's reliability in predicting stock prices.

1) *R-squared*: It computes the proportion of the variance in the actual stock prices that can be described by the predicted prices. A larger R-squared indicates that the model can explain a huge part of the variability in stock prices, pointing towards a good predictive ability.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(Z_i - \hat{Z}_i)^2}{\sum_{i=1}^{n}(Z_i - \bar{Z})^2} \tag{10}$$

2) *Mean squared error*: It represents the average of the squared differences between actual and predicted stock prices. A lesser MSE indicates that the predictions are closer to the actual values, which a sign of high accuracy.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Z_i - \hat{Z}_i)^2 \tag{11}$$

3) *Mean absolute error*: It is the average of the absolute differences between actual and predicted stock prices. It provides a candid measure of prediction accuracy without giving extra weight to larger errors.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|Z_i - \hat{Z}_i\right| \tag{12}$$

4) *Root mean square error*: It is the square-root of the averaged squared differences between actual and predicted stock prices. It helps in understanding the magnitude of errors in the same units as the original data and is more sensitive to outliers.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Z_i - \hat{Z}_i)^2} \tag{13}$$

5) *Mean absolute percentage error*: It represents the average absolute percentage error between actual and predicted stock prices. It is useful for comparing prediction accuracy across different datasets and models as it expresses error as a percentage.

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{Z_i - \hat{Z}_i}{Z_i}\right| \tag{14}$$

6) *Symmetric mean absolute percentage error*: It adjusts mean absolute percentage error to handle zero values and to ensure that overestimates and underestimates are treated symmetrically, making it more suitable for stock market prediction where stock prices can vary widely.

$$\text{sMAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\frac{|Z_i - \hat{Z}_i|}{(|Z_i| + |\hat{Z}_i|)/2} \tag{15}$$

7) *Mean bias deviation:* It measures the average bias in predictions, indicating whether predictions are generally higher or lower than actual values. In stock market prediction, a nonzero MBD implies a systematic bias in the model.

$$\text{MBD} = \frac{1}{n}\sum_{i=1}^{n}(Z_i - \hat{Z}_i) \tag{16}$$

8) *Explained variance score*: It computes the fraction of variance in the true stock prices described by the model. A higher value implies that the model accounts for more variability in stock prices. It is quite similar to R-squared.

$$\text{EVS} = 1 - \frac{\text{Var}(Z - \hat{Z})}{\text{Var}(Z)} \tag{17}$$

9) *D^2 Absolute error score*: It compares the sum of absolute errors of the model to that of a naive predictor using the mean of the actual values. It provides a normalized

TABLE II
PERFORMANCE METRICS FOR DIFFERENT MODELS OVER VARIOUS YEARS ALPHABET

Years	Model	R^2	MSE	MAE	RMSE	MAPE	sMAPE	MBD	EVS	D^2
5	LNN	0.7929	0.0766	0.2073	0.2769	0.0191	1.9202	−0.006	0.7929	0.6061
	LSTM	0.3925	1.6732	1.091	1.2935	0.038	3.8983	−0.745	0.5941	0.2393
	RNN	0.4500	1.5147	1.0445	1.2307	0.0363	3.7270	−0.957	0.7828	0.2723
10	LNN	0.9561	0.4437	0.5277	0.6661	0.0211	2.1083	0.1103	0.9573	0.8034
	LSTM	0.8574	1.4425	9.6777	1.2010	0.0358	3.8284	−0.131	0.8591	0.6396
	RNN	0.8751	1.2630	0.9294	1.1238	0.0347	3.5431	−0.748	0.9304	0.6538
15	LNN	0.9374	0.0639	0.1876	0.2528	0.0190	1.888	0.0585	0.9407	0.7853
	LSTM	0.8959	0.7901	0.7048	0.8889	0.0280	2.7803	0.0363	0.8961	0.7044
	RNN	0.7156	2.1602	1.2721	1.4697	0.0476	4.9116	−1.232	0.9154	0.4664

TABLE III
PERFORMANCE METRICS FOR DIFFERENT MODELS OVER VARIOUS YEARS APPLE

Years	Model	R^2	MSE	MAE	RMSE	MAPE	sMAPE	MBD	EVS	D^2
5	LNN	0.9718	0.4987	0.5412	0.7062	0.0288	2.8713	0.0183	0.9718	0.8473
	LSTM	0.8625	1.1413	0.9143	1.1888	0.0431	4.2068	0.3485	0.8752	0.6177
	RNN	0.8907	1.1316	0.8191	1.0637	0.0356	3.6351	−0.613	0.9270	0.6575
10	LNN	0.9748	0.4452	0.5165	0.6672	0.0277	2.7450	0.1416	0.9760	0.8543
	LSTM	0.8820	2.0910	1.2216	1.446	0.6831	6.5056	0.9967	0.9380	0.6555
	RNN	0.9643	0.6314	0.6167	0.7946	0.0319	3.1671	0.0179	0.9643	0.8260
15	LNN	0.9767	0.4114	0.4938	0.6414	0.0263	2.6097	0.0478	0.9769	0.8607
	LSTM	0.8857	2.0254	1.7888	1.4123	0.0651	6.2404	0.7684	0.9190	0.6675
	RNN	0.9471	0.9373	0.7630	0.9681	0.0407	4.0035	0.1456	0.9483	0.7848

TABLE IV
PERFORMANCE METRICS FOR DIFFERENT MODELS OVER VARIOUS YEARS BLACKROCK

Years	Model	R^2	MSE	MAE	RMSE	MAPE	sMAPE	MBD	EVS	D^2
5	LNN	0.9558	0.3863	0.4815	0.6216	0.0242	2.4357	−0.116	0.9573	0.8123
	LSTM	0.8444	1.3605	0.9425	1.1664	0.0496	4.8086	0.6727	0.8961	0.6326
	RNN	0.9240	0.6641	0.5989	0.8149	0.0304	3.0291	0.0413	0.9242	0.7665
10	LNN	0.9201	0.7056	0.6075	0.8400	0.0328	3.2733	0.0469	0.9204	0.7517
	LSTM	0.5267	4.1849	1.7831	2.0457	0.0989	9.2620	1.7249	0.8632	0.2714
	RNN	0.9278	0.6379	0.6258	0.7987	0.0346	3.3991	0.3672	0.9431	0.7442
15	LNN	0.8669	1101.9	28.344	33.195	0.0404	4.1536	−25.99	0.9485	0.6346
	LSTM	0.8897	1.1444	0.9876	1.2017	0.0463	4.4944	0.6250	0.9195	0.6708
	RNN	0.9307	0.9071	0.7118	0.9524	0.0297	3.008	−0.301	0.9376	0.7627

measure of accuracy, with values closer to 1 indicating better performance.

$$D^2 = 1 - \frac{\sum_{i=1}^{n} |Z_i - \hat{Z}_i|}{\sum_{i=1}^{n} |Z_i - \bar{Z}|}. \tag{18}$$

B. Comparison of Liquid Neural Networks With Other Models

Alphabet: Considering various time frames of 5, 10, and 15 years we have noted all the results individually as shown in Table II, here we observed that the LNN outperformed the other two most commonly used algorithms viz. RNN and LSTM, the R^2 values for the dataset were observed to increase with the size of the dataset, from 0.9718 to 0.9767. We also observed the other performance evaluation factors, i.e., MSE, MAE, RMSE, MAPE, sMAPE, MBD, EVS, and D^2 were observed to be declining toward more positive result, there were no anomaly observed here in this data as it can be noticed from the Table II that errors we reduced with the increase in dataset.

Apple: Table III highlights the evaluation results for Apple stock market data across the time span of past 5, 10, and 15 years. Liquid Neural Networks showed the best results in comparison to LSTM & RNN. Over the period of 5 years, the R^2 values obtained for the models were poor, although LNN (0.7929) still outperforms LSTM (0.3925) and RNN (0.4500). The D^2 score for LNN noticeably increased over the years from 0.6061 for 5 years to 0.7853 for 15 years. The minimum symmetric mean absolute percentage value of 1.888 was observed for data of 15 years. Other metrics are also inclined towards positive results for LNN. These sets of results also showed a lack of anomalous values.

BlackRock: Table IV shows the results observed for Blackrock Stock for three different dataset size, i.e., for 5 to 15 years,

TABLE V
PERFORMANCE METRICS FOR DIFFERENT MODELS OVER VARIOUS YEARS JPM

Years	Model	R^2	MSE	MAE	RMSE	MAPE	sMAPE	MBD	EVS	D^2
5	LNN	0.9763	0.5311	0.5488	0.7288	0.0256	2.5688	−0.103	0.9768	0.8668
	LSTM	0.9439	1.2595	0.9069	1.1223	0.04132	4.137	−0.284	0.9475	0.7799
	RNN	0.9392	1.3648	0.8787	1.1682	0.0369	3.7798	−0.689	0.9603	0.7867
10	LNN	0.9408	1.400	1.0339	1.1833	0.0613	5.8686	0.9586	0.9796	0.7211
	LSTM	0.8988	2.3946	1.2880	1.5474	0.07015	6.7986	0.4287	0.9066	0.6526
	RNN	0.9647	0.8344	0.7058	0.9134	0.0371	3.6665	0.0631	0.9649	0.8096
15	LNN	0.9465	0.739	0.6688	0.8601	0.0340	3.3198	0.4920	0.9640	0.7741
	LSTM	0.8878	1.5517	1.0409	1.2456	0.0538	5.1816	0.7819	0.9320	0.6457
	RNN	0.9336	0.9172	0.7174	0.9577	0.0316	3.2145	−0.504	0.9520	0.7577

TABLE VI
PERFORMANCE METRICS FOR DIFFERENT MODELS OVER VARIOUS YEARS IBM

Years	Model	R^2	MSE	MAE	RMSE	MAPE	sMAPE	MBD	EVS	D^2
5	LNN	0.9286	2.1614	1.1146	1.4701	0.0499	5.1047	−0.784	0.9489	0.7682
	LSTM	0.9238	2.3069	1.1169	1.5188	0.05088	5.2251	−0.723	0.9410	0.7678
	RNN	0.8910	3.3002	1.3809	1.8166	0.0595	6.2046	−1.283	0.9453	0.7129
10	LNN	0.9668	1.0297	0.7540	1.0147	0.0474	4.6310	0.4594	0.9736	0.8179
	LSTM	0.9043	2.9685	1.4082	1.7229	0.0821	8.6325	−1.179	0.9491	0.6599
	RNN	0.9509	1.5227	0.8798	1.2339	0.0493	4.9914	−0.442	0.9572	0.7875
15	LNN	0.9754	0.5381	0.5527	0.7335	0.0407	4.1157	−0.235	0.9780	0.8260
	LSTM	0.9443	1.222	0.8681	1.1055	0.06542	6.4755	0.0782	0.9446	0.7267
	RNN	0.9752	0.5426	0.5398	0.7366	0.0415	4.0849	0.1845	0.9768	0.7830

TABLE VII
FEATURE SELECTION WITH BORUTA ALGORITHM

Company	Years	Open	High	Low	Adj Close	Volume	Year	Month	Day	Close(Y)
Alphabet	**5**	✓	✓	✓	✓	✗	✗	✓	✗	✓
	10	✓	✓	✓	✓	✗	✗	✓	✗	✓
	15	✓	✓	✓	✓	✗	✗	✓	✗	✓
Apple	**5**	✓	✓	✓	✓	✗	✗	✓	✗	✓
	10	✓	✓	✓	✓	✗	✗	✓	✗	✓
	15	✓	✓	✓	✓	✗	✗	✓	✗	✓
BlackRock	**5**	✓	✓	✓	✓	✗	✗	✗	✗	✓
	10	✗	✓	✓	✓	✗	✗	✗	✗	✓
	15	✗	✓	✓	✓	✗	✗	✗	✗	✓
JPM	**5**	✓	✓	✓	✓	✗	✗	✗	✗	✓
	10	✓	✓	✓	✓	✗	✗	✗	✗	✓
	15	✓	✓	✓	✓	✗	✗	✗	✗	✓
IBM	**5**	✓	✓	✓	✓	✗	✗	✗	✗	✓
	10	✓	✓	✓	✓	✗	✗	✗	✗	✓
	15	✗	✓	✗	✗	✗	✗	✗	✗	✓

observing all the values of LNN, LSTM and RNN we can say the LNN gives the R-squared value with 95.58% for 5 years data followed by RNN 93.07% for large dataset, Here we observe some anomaly in the for LSTM where we observed a high fall in R-square value from 5 years dataset to 10 years dataset, hence overall we can observe that LNN outperformed LSTM and RNN for Blackrock Historical data.

JPM: Table V shows the result noted for JP Morgan for three different years, i.e., past 5, 10, and 15 years historical data, observing the result for LNN, RNN, and LSTM, we can say that LNN with a R-squared value of 0.9763 performs the best and the fastest followed by RNN with the best value of 0.9647 although there is an anomaly in years it gave better result for larger data. Hence we can say that LNN gives the best result and faster processing from RNN and LSTM.

IBM: Table VI shows the result for IBM historical data of 5, 10, and 15 years, after observing the results of LNN, RNN, and LSTM we got the best result for LNN with a R-square value of 0.9754 followed by RNN 0.9752, here we observe the same anomaly observed in JPM where LSTM gave better R-square value for 5 years data as compared to RNN but with the increase in dataset the RNN gave better result than LSTM but LNN outperformed in all the type of dataset, comparison of all the algorithm are shown in Table IX.

TABLE VIII
STATISTICAL RESULTS OF LNN RESULTS WITH BORUTA ALGORITHM

Company	Years	R^2	MSE	MAE	RMSE	MAPE	sMAPE	MBD	EVS	D^2
Alphabet	5	0.9586	0.0576	0.1705	0.2400	0.0197	1.9833	−0.046	0.9601	0.8055
	10	0.9777	0.0529	0.1794	0.2300	0.0234	2.3515	−0.035	0.9783	0.8619
	15	0.9801	0.0473	0.1675	0.2176	0.0221	2.2056	−0.006	0.9801	0.8711
Apple	5	0.8589	0.3884	0.4857	0.6232	0.0176	1.7524	0.1953	0.8728	0.6615
	10	0.9647	0.0478	0.1703	0.2188	0.0174	1.7335	0.0551	0.9670	0.8270
	15	0.9687	0.2370	0.3821	0.4869	0.0151	1.5171	−0.048	0.9690	0.8397
BlackRock	5	0.9737	95.3733	7.4781	9.7659	0.0109	1.0910	3.3070	0.9768	0.8571
	10	0.8738	682.7647	22.8556	26.1297	0.0331	3.3824	−22.15	0.9645	0.6157
	15	0.9551	0.5873	0.6143	0.7664	0.0282	2.7741	0.3944	0.9670	0.7953
JPM	5	0.9835	7.5256	2.1145	2.7432	0.0129	1.2984	−1.099	0.9861	0.8861
	10	0.9834	11.5583	2.5593	3.3997	0.0189	1.8971	−0.261	0.9835	0.8730
	15	0.9762	12.4810	2.6818	3.5328	0.0191	1.906	−0.008	0.9762	0.8529
IBM	5	0.9840	7.9759	1.9004	2.8241	0.0122	1.2235	−0.479	0.9845	0.9029
	10	0.9734	13.6536	2.9567	3.6950	0.0209	2.1149	−2.235	0.9831	0.8245
	15	0.9844	5.3790	1.5969	2.3192	0.0113	1.1389	0.0159	0.9844	0.8727

TABLE IX
COMPARISON TABLE WITH EXISTING MODELS

Company	Years	R^2	MSE	MAE	RMSE
LNN	0.8954	0.1947	0.3075	0.3986	0.0197
LSTM	0.7152	1.301	0.9211	1.127	0.0339
RNN	0.680	1.6459	1.082	1.274	0.0395
LNN with feature selection	0.9721	0.0526	0.1724	0.2292	0.0217

C. Boruta Feature Selection

Today, the accuracy of the stock market predictions plays a significant role in helping stock traders make a conscious decision regarding the trade. Therefore, it is important for analysts to know which features contribute towards that decision and which do not. Our work tactically employs Boruta feature selection algorithm to filter out and identify these most important and effective features that contribute towards prediction. By doing this, it is ensured that only the most significant features are brought into play while training our model. Table I discusses the features selected by Boruta algorithm and hence, that contribute towards the stock market prediction.

During Boruta feature selection, few points are noticed. Closing prices of stocks are always selected by the Boruta method, which makes quite sense as it is the value the model must predict. Highest price of stocks is a contributing factor in all the datasets which means that the maximum price buyers were willing to pay during the day contributes quite significantly towards predicting close value. Quite interestingly, year and date are not contributing factors for any of the stock market data and month only contributes for companies Alphabet and Apple, indicating that date does not play much of an important role in comparison to features such as highest prices of the stocks.

The second most significant factors, as noticed in the stock market data, are the lowest prices and the adjusted closing values of the stocks. This indicates that minimum price traders will accept and the actual reflection of a stock's value based on companies' decisions play an important role in predicting the closing price values. Surprisingly, the trading volume of the stocks is not a deciding factor for any of the companies' stocks indicating the high or low activity is not important for final predictions.

D. LNN With Boruta Algorithm

The average results of our approach showed a noticeable improvement upon compared to using just LNN, as seen in Table VIII. After using Boruta Algorithm, we noticed 2.19% increase in R-squared value on an average, going from to 0.9391 to 0.9597, showing high explanatory ability of the approach. Increase in the prediction ability is noticed by 5.25% increase in D^2 score, going from 0.782 to 0.8231. Our approach is able to describe on the variance in the stock market data well, as demonstrated by 1.90% increase in average explained variance score, from 0.9508 to 0.9689. Lower bias in our model is observed as backed up the mean bias deviation score (–1.493) closer to zero on an average across all companies, contrasted by –1.660 mean deviation score achieved by just LNN. Both symmetric mean absolute percentage error metrics and mean absolute percentage error, on an average, show a decrease of 43.07% (0.0189 to 0.0332) and 42.81% (3.3076 to 1.8913), respectively, showing better more balanced and better predictions. It has been noticed that our approach gives more desirable results as compared to other approaches.

Although, an anomaly is observe when our hybrid model is trained on past 5-year stock data of companies Apple, giving poorer results as compared LNNs only. The R^2 value, in this

case, have decreased from 0.9718 to 0.8589. Another anomaly is observed for past 10-year stock data of BlackRock company where the R^2 values have again decreased from 0.9201 to 0.8738. However, no such outliers have been observed for the past 15 years data, suggesting that huge amount of stock market data is needed for training the model.

With much more flexible structure, LNN captures complex, nonlinear relationships in the data while Boruta generalizes by reducing overfitting. Moreover, Boruta aids in interpretability by focusing on feature importance, thus understanding the reason driving prediction which is helpful in the applications of finance. While possessing these strengths, challenges remain to accept. While Boruta is powerful, it is computationally and memory taxing (which can increase resource usage) when it comes to using them on large datasets. Likewise, LNNS might need precise adjustment of hyperparameters, like connectivity configurations, for reaching peak performance. While Boruta guarantees the selection of pertinent features, there's a chance of omitting mildly correlated variables that might aid in capturing subtle market trends. Additionally, the inherently intricate characteristics of LNNs can obscure their decision-making processes, despite the interpretability provided by Boruta. Finally, the adaptability of LNNs is both a major advantage as well as something that should be consistently monitored to ensure stability in highly volatile markets. By employing efficient computational strategies and conducting regular evaluations, these considerations can be effectively addressed, ensuring that the potential of this approach for stock market predictions is maximized.

V. Conclusion

We investigated the stock price of five large cap sectors using our novel approach of a LNN with the Boruta feature selection algorithm and with the help of ODE solvers in LTC, we got more accurate results. Upon comparing it with LSTM and RNN, we noted LNN relatively took less time to train as well as achieved the best accuracy for the past 15-year dataset followed by the RNN. We also observed that with increasing the size of the dataset, the accuracy increased for all algorithms except a few anomalies in LSTM and RNN, where LSTM gave a better result than RNN for small datasets of JPMorgan and IBM.

We also observed that the results before feature selection were comparatively less than the ones obtained after using feature selection where upto 4.5% improvement was achieved, here certain columns such as the day, volume, and year were ranked as not-significantly important as it is shown in the Table VII. Overall, our approach of using Boruta Feature Selection Algorithm with LNNs outperformed the conventional methods, viz., LSTM and RNN irrespective of the size of dataset and also in processing time with the best accuracy being 98.35 in the scale of 100. As part of the future works, exploration of our approach with for larger stock market data is possible. Also, an integration of explainable AI frameworks to provide more interpretability and transparency can be implemented.

References

[1] R. Chiong, Z. Fan, Z. Hu, and S. Dhakal, "A novel ensemble learning approach for stock market prediction based on sentiment analysis and the sliding window method," *IEEE Trans. Comput. Soc. Syst.*, vol. 10, no. 5, pp. 2613–2623, Oct. 2023.

[2] S. Li, Z. Lin, Z. Xiao, and J. Ma, "The use of GARCH-neural network model for forecasting the volatility of bid-ask spread of the Chinese stock market," in *Proc. Int. Conf. Manage. Sci. Eng. 19th Annu. Conf. Proc.*, IEEE, 2012, pp. 1899–1903.

[3] K. Uhle, "Predicting stock market liquidity using neural networks," Master's Thesis, 2020.

[4] P. Gajjar, A. Saxena, K. Acharya, P. Shah, C. Bhatt, and T. Nguyen, "Liquidt: Stock market analysis using liquid time-constant neural networks," *Int. J. Inf. Technol.*, vol. 16, no. 2, pp. 909–920, 2024.

[5] J. Ohana, S. Ohana, E. Benhamou, D. Saltiel, and B. Guez, "Explainable AI (XAI) models applied to the multi-agent environment of financial markets," *Explainable Transparent AI Multi-Agent Syst.: 3rd Int. Workshop, EXTRAAMAS 2021, Virtual Event*, May 3–7, 2021, pp. 189–207.

[6] T. B. Çelik and E. Bulut, "Extending machine learning prediction capabilities by explainable AI in financial time series prediction," *Appl. Soft Comput.*, vol. 132, 2023, Art. no. 109876.

[7] Y. Fukuchi and S. Yamada, "Dynamic explanation selection towards successful user-decision support with explainable AI," 2024, *arXiv:2402.18016*.

[8] W. Freeborough and T. Zyl, "Investigating explainability methods in recurrent neural network architectures for financial time series data," *Appl. Sci.* vol. 12, no. 3, p. 1427, 2022.

[9] X. Pang, Y. Zhou, P. Wang, W. Lin, and V. Chang, "An innovative neural network approach for stock market prediction," *J. Supercomput.*, vol. 76, no. 1, pp. 2098–2118, 2020.

[10] J. Chen, L. Song, M. Wainwright, and M. Jordan, "Learning to explain: An information-theoretic perspective on model interpretation," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 883–892.

[11] Y. Li, H. Xue, S. Wei, R. Wang, and F. Liu, "A machine learning approach for investigating the determinants of stock price crash risk: Exploiting firm and ceo characteristics," *Systems*, vol. 12, no. 5, pp. 143, 2024.

[12] A. Agarwal, A. Bhatia, A. Malhi, P. Kaler, and H. Pannu, "Machine learning based explainable financial forecasting," in *Proc. 4th Int. Conf. Comput. Commun. Internet (ICCCI)*, IEEE, 2022, pp. 34–38.

[13] S. Gite, H. Khatavkar, K. Kotecha, S. Srivastava, P. Maheshwari, and N. Pandey, "Explainable stock prices prediction from financial news articles using sentiment analysis," *PeerJ Comput. Sci.*, vol. 7, no. 1, pp. 340, 2021.

[14] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, "Liquid time-constant networks," *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 9, pp. 7657–7666, 2021.

[15] J. Li, Y. Liu, H. Gong, and X. Huang, "Stock price series forecasting using multi-scale modeling with Boruta feature selection and adaptive denoising," *Appl. Soft Comput.* vol. 154, 2024, Art. no. 111365.

[16] K. Pawar, R. Jalem, and V. Tiwari, "Stock market price prediction using LSTM RNN," in *Proc. Emerg. Trends Expert Appl. Secur.: Proc. ICETEAS 2018*, Springer Singapore, 2019, pp. 493–503.

[17] D. Nelson, A. Pereira, and R. Oliveira, "Stock market's price movement prediction with lstm neural networks," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, IEEE, 2017, pp. 1419–1426.

[18] M. Kumbure, C. Lohrmann, P. Luukka, and J. Porras, "Machine learning techniques and data for stock market forecasting: A literature review," *Expert Syst. Appl.*, vol. 197, 2022, Art. no. 116659.

[19] H. Widiputra, A. Mailangkay, and E. Gautama, "Multivariate CNN-LSTM model for multiple parallel financial time-series prediction," *Complexity*, vol. 2021, no. 1, pp. 1–14, 2021.

[20] S. Hansun and J. Young, "Predicting lq45 financial sector indices using RNN-LSTM," *J. Big Data*, vol. 8, no. 1, pp. 104, 2021.

[21] X. Zhang, X. Liang, A. Zhiyuli, S. Zhang, R. Xu, and B. Wu, "At-lstm: An attention-based lstm model for financial time series prediction," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 569, no. 5, p. 052037, 2019.

[22] J. Nielsen, D. Sornette, and M. Raissi, "Deep lppls: Forecasting of temporal critical points in natural, engineering and financial systems," 2024, *arXiv:2405.12803*.

[23] M. Sethi, A. Sadhu, K. Pahwa, S. Nagpal, and T. Sethi, "Variance of twitter embeddings and temporal trends of covid-19 cases," 2021, *arXiv:2110.00031*.

[24] W. Chen, C. Yeo, C. Lau, and B. Lee, "Leveraging social media news to predict stock index movement using rnn-boost," *Data Knowl. Eng.*, vol. 118, pp. 14–24, 2018.

[25] B. Pramod and M. Pm, "Stock price prediction using LSTM," *Test Eng. Manage.*, vol. 83, pp. 5246–5251, 2021.
[26] S. Wu, Y. Liu, Z. Zou, and T. Weng, "S_I_LSTM: Stock price prediction based on multiple data sources and sentiment analysis," *Connect. Sci.*, vol. 34, no. 1, pp. 44–62, 2022.
[27] Y. Liu, Z. Qin, P. Li, and T. Wan, "Stock volatility prediction using recurrent neural networks with sentiment analysis," in *Proc. Adv. Artif. Intell.: From Theory Pract.: 30th Int. Conf. Ind. Eng. Other Appl. Appl. Intell. Syst., IEA/AIE 2017*, vol. Proceedings, Part I 30, (Arras, France), Springer International Publishing, 2017, pp. 192–201.
[28] F. Moodi, A. Jahangard-Rafsanjani, and S. Zarifzadeh, "A cnn-lstm deep neural network with technical indicators and sentiment analysis for stock price forecastings," in *Proc. 20th CSI Int. Symp. Artif. Intell. Signal Process. (AISP)*, IEEE, 2024, pp. 1–6.
[29] M. Khalid, I. Ashraf, A. Mehmood, S. Ullah, M. Ahmad, and G. Choi, "GBSVM: Sentiment classification from unstructured reviews using ensemble classifier," *Appl. Sci.*, vol. 10, no. 8, pp. 2788, 2020.
[30] A. Md, S. Kapoor, C. AV, A. Sivaraman, K. Tee, H. Sabireen, and N. Janakiraman, "Novel optimization approach for stock price forecasting using multi-layered sequential LSTM," *Appl. Soft Comput.* vol. 134, 2023, Art. no. 109830.
[31] S. Verma, S. P. Sahu, and T. Sahu, "Stock market forecasting with different input indicators using machine learning and deep learning techniques. A review," *Eng. Lett.*, vol. 31, no. 3, pp. 213–229, 2023.
[32] T. Strader, J. Rozycki, T. Root, and Y. Huang, "Machine learning stock market prediction studies: Review and research directions," *J. Int. Technol. Inf. Manage.*, vol. 28, no. 4, pp. 63–83, 2020.
[33] S. Yu, S. Chu, Y. Chan, and C. Wang, "Share price trend prediction using CRNN with LSTM structure," *Smart Sci.*, vol. 7, no. 3, pp. 189–197, 2019.
[34] X. Zhong and D. Enke, "Predicting the daily return direction of the stock market using hybrid machine learning algorithms," Financial Innov., vol. 5, no. 1, pp. 1–20, 2019.
[35] A. Samarawickrama and T. Fernando, "A recurrent neural network approach in predicting daily stock prices an application to the Sri Lankan stock market," in *Proc. IEEE Int. Conf. Ind. Inf. Syst. (ICIIS)*, IEEE, 2017, pp. 1–6.
[36] D. Sheth and M. Shah, "Predicting stock market using machine learning: best and accurate way to know future stock prices," *Int. J. Syst. Assur. Eng. Manage.*, vol. 14 no. 1, pp. 1–18, 2023.
[37] K. Kumar and M. Haider, "Enhanced prediction of intra-day stock market using metaheuristic optimization on RNN–LSTM network," *New Gener. Comput.*, vol. 39, no. 1, pp. 231–272, 2021.
[38] S. Albahli, A. Irtaza, T. Nazir, A. Mehmood, A. Alkhalifah, and W. Albattah, "A machine learning method for prediction of stock market using real-time twitter data," *Electronics*, vol. 11 no. 20, pp. 3414, 2022.
[39] H. Niu, "A method to increase the analysis accuracy of stock market valuation: A case study of the Nasdaq index," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 1, pp. 732–743, 2024.
[40] F. Rundo, F. Trenta, A. Stallo, and S. Battiato, "Machine learning for quantitative finance applications: A survey," *Appl. Sci.* vol. 9, no. 24, pp. 5574, 2019.
[41] S. Gao, B. Lin, and C. Wang, "Share price trend prediction using crnn with lstm structure," in *Proc. Int. Symp. Comput., Consum. Control (IS3C)*, IEEE, 2018, pp. 10–13, IEEE.
[42] H. Gunduz, "An efficient stock market prediction model using hybrid feature reduction method based on variational autoencoders and recursive feature elimination," *Financial Innov.*, vol. 7, no. 1, p. 28, 2021.
[43] V. Mahajan, S. Thakan, and A. Malik, "Modeling and forecasting the volatility of Nifty 50 using GARCH and RNN models," *Economies*, vol. 10, no. 5, pp. 102, 2022.
[44] Q. Gao, "Stock market forecasting using recurrent neural network," Doctoral dissertation, University of Missouri–Columbia, 2016.
[45] M. Chahin et al., "Robust flight navigation out of distribution with liquid neural networks," *Sci. Robot.*, vol. 8, no. 77, pp. 8892, 2023.
[46] S. Khatri and A. Srivastava, "Using sentimental analysis in prediction of stock market investment," in *Proc. 5th Int. Conf. Rel., Infocom Technol. Optim. (Trends Future Directions) (ICRITO)*, IEEE, 2016, pp. 566–569.
[47] S. Verma, S. Sahu, and T. Sahu, "Wavelet decomposition-based multi-stage feature engineering and optimized ensemble classifier for stock market prediction," in *Eng. Econ.*, vol. 69, no. 3, pp. 1–26, 2024.
[48] Yahoo Finance - Business Finance Stock Market News. Accessed: May 24, 2024. [Online]. Available: https://finance.yahoo.com/quote/

G. Joshita Reddy was born in Andhra Pradesh, India, in 2002. She received the bachelor's degree in computer science with specialisation in data science from Vellore Institute of Technology, Vellore, India, in 2024.

Currently, she is working as an Business Analyst. Her research interests include neural networks and artificial intelligence.

Rahul Kumar Sahani was born in Uttar Pradesh, India, in 2003. He received the B.Tech degree in computer science from Vellore Institute of Technology, Vellore, India, in 2024.

Currently, he is working as an Software Engineer. He has published research articles in the fields of Machine Learning and AI. He was selected among 60 students nationwide to present his idea at the Indian Space Research Organisation (ISRO). His research interests include machine learning, artificial intelligence, and cybersecurity.

S. P. Raja was born in Tamilnadu, India. He received the B.Tech degree in information technology from Dr. Sivanthi Aditanar College of Engineering, Tiruchendur, India, in 2007, the M.E. degree in computer science and engineering and the Ph.D. degree in Image processing from Manonmaniam Sundaranar University, Tirunelveli, India, in 2010 and 2016, respectively.

Currently, he is working as an Associate Professor with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamilnadu, India.